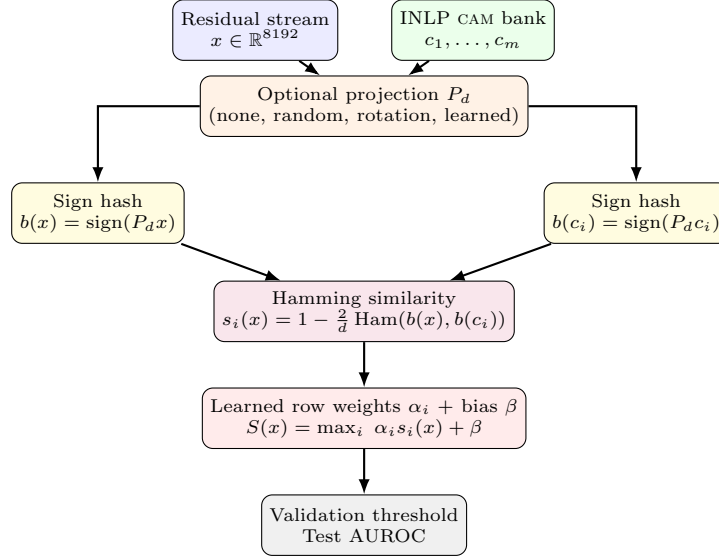


Binary CAM Matching for Efficient Transformer Deception Detection

Goal. The question is whether a deception detector built from transformer residual streams can be implemented with *binary* representations and simple Hamming-distance logic rather than full-precision Euclidean or dot-product matching. Concretely, each example produces a residual stream $x \in \mathbb{R}^{8192}$, and a bank of deception-related directions $c_1, \dots, c_m$ is extracted with iterative null-space linear projection (INLP). INLP repeatedly trains a linear probe, stores the normalized probe direction as a new CAM row, projects that direction out, and repeats, producing multiple complementary deception directions instead of one probe.



In the simplest direct-sign system, there is no projection and the binary code is just $b(x) = \text{sign}(x)$ and $b(c_i) = \text{sign}(c_i)$. Similarity is then normalized Hamming agreement,

$$s_i(x) = 1 - \frac{2}{d} \text{Ham}(b(x), b(c_i)) = \frac{1}{d} b(x)^\top b(c_i),$$

followed by a learned positive weight on each CAM row and a winner-take-all readout. The row weights and bias are relearned at every code length d , but the extracted CAM directions are fixed.

What the sweep is testing. The code-length sweep asks how small d can be before binary matching loses too much ranking quality. This was evaluated with several projection choices before taking signs: random projection, random coordinate selection, magnitude-based coordinate selection, random rotation plus truncation, and a learned hashing matrix. The key methodological issue is interpretability. A strong result is one in which *label-free* or structurally simple projections preserve AUROC, because that supports the claim that the binary CAM mechanism itself is doing useful work. A weaker result is one in which a learned projection recovers performance by itself, because then the learned matrix may be doing most of the classification.

Current takeaways from the hamming experiments.

- **Learned down projection / learned hash:** this gives the best-looking Hamming results, but it is the least satisfying hardware story. The learned projection can absorb much of the task structure, so the final success may say more about the projection than about binary CAM matching. It is also expensive: in the square case it effectively requires storing and applying an 8192×8192 matrix.
- **Random or other fixed down projections:** these are cleaner because they do not learn the task, but they still require an $8192 \times d$ matrix multiply at inference time. They are best viewed as a middle ground: useful for probing compressibility, but not the cleanest deployment path.
- **Direct sign hash:** this is the cleanest systems result. There is no projection matrix, no extra matrix multiplication, and only the binary CAM rows need to be stored. If the direct-sign variant gives competitive AUROC, it is the most compelling option for a hardware CAM implementation because it reduces inference to sign extraction, XOR/popcount-style Hamming matching, and a small weighted WTA readout.

How this relates to DReX and LongSight. DReX introduces *Sign Concordance Filtering*: query and database embeddings are reduced to sign bits, compared with XOR and popcount, and vectors with too few matching signs are filtered before full-precision scoring [?]. LongSight repurposes that idea for long-context attention, storing KV-cache entries in compute-enabled CXL memory and filtering old keys using sign-bit agreement before exact top- k attention scoring [?]. The conceptual overlap with this project is direct: all three systems use binary sign patterns as a cheap proxy for angular similarity.

The difference is *what the binary stage is for*. In DReX and LongSight, sign matching is a *filtering* stage before a full-precision second stage. In this project, sign matching is the classifier itself: Hamming similarity to a small bank of INLP-extracted concept directions is turned directly into a deception score. LongSight also reports that raw sign bits can be poorly balanced and uses Iterative Quantization (ITQ), a learned rotation that improves binary filtering without supervision. That maps naturally onto your sweep: random rotations are a conservative unsupervised analogue, while the learned hash matrix is a stronger but less interpretable analogue.

	DReX	LongSight	This project
Vectors	Query/document embeddings	Attention queries/keys	Residuals / INLP CAM rows
Binary stage	Candidate filtering	KV filtering	Final classifier similarity
Second stage	Exact dense retrieval	Full-precision top- k attention	Weighted WTA only
Primary metric	Recall / throughput	Perplexity / throughput	AUROC / accuracy
Best systems story	Cheap filter before ENNS	Cheap filter before attention	Direct sign hash, no projection

Bottom line. Right now, the most informative comparison is not simply “which method gets the highest AUROC,” but rather “which method preserves AUROC *without* moving the work into a large learned projection.” On that criterion, direct sign hashing is the most important result to track, because it is the variant that most cleanly tests the hardware-relevant hypothesis.

References

- [1] D. Quinn et al., “DReX: Accurate and Scalable Dense Retrieval Acceleration via Algorithmic-Hardware Codesign,” *ISCA*, 2025.
- [2] D. Quinn et al., “LongSight: Compute-Enabled Memory to Accelerate Large-Context LLMs via Sparse Attention,” *MICRO*, 2025.